Design patterns and backend architecture

Chapter 8: Four worked demonstrations: prove the choice, not just the name

Practice designs, not claims about an employer.

8. Four worked demonstrations: prove the choice, not just the name

These are **illustrative Java 17+/Spring-style examples**, not complete payment code. They omit authentication plumbing, currency precision rules, full persistence mapping, and vendor SDK contracts. They are practice designs, **not implementations attributed to a real employer**.

Demonstration A: change a fee rule without changing submission orchestration

Starting problem: One `submit()` method contains fee switches, builds external requests, and saves payment rows. Adding a fee policy means editing and retesting unrelated provider code.

```
import java.math.BigDecimal;

interface FeePolicy {
    BigDecimal calculate(PaymentDraft payment);
}

record PaymentDraft(String tenantId, BigDecimal amount, String currency) {}

final class FixedFeePolicy implements FeePolicy {
    public BigDecimal calculate(PaymentDraft payment) {
        return new BigDecimal("1.50"); // Illustrative rule only.
    }
}

interface PaymentStore {
    void save(PaymentDraft payment, BigDecimal fee);
}

final class SubmitPaymentUseCase {
    private final FeePolicy feePolicy;
    private final PaymentStore store;

    SubmitPaymentUseCase(FeePolicy feePolicy, PaymentStore store) {
        this.feePolicy = feePolicy;
        this.store = store;
    }

    void accept(PaymentDraft payment) {
        BigDecimal fee = feePolicy.calculate(payment);
        store.save(payment, fee);
    }
}
```

Reasoning: SRP gives fee decisions and persistence separate reasons to change. Strategy defines interchangeable fee rules; composition injects the choice. DIP makes the use case depend on `PaymentStore`, not JDBC. **Evidence to show in an interview:** A new fee-policy implementation is tested

without changing `SubmitPaymentUseCase`; an in-memory store validates what the use case passes to persistence. **Tradeoff:** This adds interfaces; for a single fixed fee rule in a short-lived tool, direct code may be simpler. **Not production ready:** Currency-specific scale/rounding, validation, authorization, audit, idempotency and transactions remain to be designed.

Demonstration B: protect accepted payments from a lost response

Scenario: Two clients retry the same instruction while the first API response is lost. A process-local `synchronized` block is not enough when multiple instances accept requests.

```
Request (tenantId, idempotencyKey, normalizedPayload)
     |
     v
Single local DB transaction
     +-- insert instruction with durable ID
     +-- insert idempotency record (UNIQUE tenantId, key)
     +-- insert outbox record (UNIQUE eventId)
     |
     v
Commit -> return instruction ID + ACCEPTED
             |
             v
          Relay -> broker -> deduplicating consumer
```

Detailed decision: Store a stable payload fingerprint with the key. A repeat with the same key and equivalent payload returns the previously recorded instruction; a repeat with a different payload is a conflict. Let a **database uniqueness constraint** decide simultaneous inserts, then read the winner in a *new, valid transaction* after a uniqueness conflict. Do not rely on `check-then-insert` without a unique constraint. Agree on key retention with the business.

How to prove it: Run two concurrent requests from different instances with one `(tenant, key)`; verify exactly one instruction and outbox record. Crash after commit but before HTTP response; a repeat returns the same ID. Crash after publish but before relay checkpoint; a duplicate event is possible but consumer effects must be idempotent. **Tradeoff:** Durable dedup records and relay monitoring are operational costs. This does not prevent a second provider charge unless the *downstream submission path* also uses safe provider-side idempotency or reconciliation.

Demonstration C: distinguish Strategy, State, Command, and Event

A single payments workflow can use these patterns for **different questions**:

How is the fee chosen?: Strategy: choose CrossBorderFeePolicy; test correct policy selection.

What may happen next?: State: SETTLED cannot return to CREATED; test invalid transitions.

What action is requested?: Command: ApprovePayment(id, actor); test unauthorized commands have no effect.

What already happened?: Event: PaymentApproved(id, time); test duplicate consumers are safe.

Why this demonstrates understanding: A strategy does not express payment lifecycle, and an event should not be emitted as if approval happened before the approval transaction commits. Using one word—"event"—for both a request and a completed fact hides critical failure handling.

Demonstration D: choose the smallest correct architecture

One team, one DB: Start with a layered modular monolith. Split only if independent deployments or scaling justify the cost.

Changing clearing providers: Use a port and adapter inside the service. A new service deployment is not necessary.

DB change must cause a broker event: Use a transactional outbox; a saga is not a substitute for atomic local writes.

Cross-service compensations: Use a saga with explicit state only when multiple independent services own the steps.

Dashboard reads differ from writes: Measure query/index limits before adding a separate CQRS projection.

Defend the choice: State the ownership boundary, consistency guarantee, worst failure case, measurable requirement, and what complexity you decline to introduce. "Microservices scale better" is not a sufficient justification by itself.

Five-minute self-check: answer with a counterexample

1. A new payment channel appears: why might a switch still be better than Strategy plus Factory?
2. A subclass compiles but throws for a documented parent operation: which promise was violated?
3. A Spring method calls its own `@Transactional` method: what boundary was bypassed?
4. An outbox relay publishes one event twice: where is the duplicate business effect prevented?
5. A provider submission times out: why is retry not automatically safe?
6. A reconciliation dashboard is slow: why test indexes/queries before adopting CQRS?

Suggested answer shape for each: "The observed problem is __. This mechanism helps because __. A simpler alternative is __. I would prove the need with __. Its failure mode is __." A good answer can also conclude that **no pattern change is justified yet**.

Answer guide: (1) Use a switch when variation is small and stable; abstraction costs more than it saves. (2) LSP: the subtype broke its promised behavior. (3) A self-call bypassed Spring proxy advice in standard proxy mode. (4) At the consumer, using a durable event ID or business key and an atomic effect/deduplication record. (5) The provider may have committed the payment even though the response was lost. (6) CQRS adds projection updates and consistency lag, so first measure whether simpler queries meet the requirement.

Runnable Java drills (no extra libraries)

Three self-contained Java 17+ files are included in the `examples/` folder, so you can see a passing behavior rather than only reading snippets:

- `FeeStrategyDemo.java`: replace the fee policy without changing the processor (Strategy, SRP, composition).
- `PaymentStateDemo.java`: permit only valid payment transitions (encapsulation, State).
- `IdempotencyToyDemo.java`: return one instruction ID for a repeated same-key request and reject a changed payload. **This deliberately uses in-memory storage and is not sufficient for a real distributed payment API.**

Run from the `examples` folder:

```
javac FeeStrategyDemo.java PaymentStateDemo.java IdempotencyToyDemo.java
java FeeStrategyDemo
java PaymentStateDemo
java IdempotencyToyDemo
```

All three were compiled and executed successfully during preparation. Use the source files to change one input, break one assertion, and explain which contract failed.

Official references for implementation-sensitive claims

- Spring: `@Transactional` and proxy limitations
- AWS: Hexagonal architecture / ports and adapters
- AWS: Transactional outbox
- Microsoft: Saga
- Microsoft: Cache-Aside
- Microsoft: Circuit Breaker

These links support the architectural *mechanisms* described above. The practice-specific business rules and designs are teaching examples, not facts about any employer or vendor.